Selection Sort

Resumo completo — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercício 6; partes dos Exercícios 1 e 7).

Propriedade	Valor
Estratégia	**seleciona** o menor elemento da parte não ordenada e o coloca na próxima posição
Melhor caso	$\Theta(n^2)$
Caso médio	$\Theta(n^2)$
Pior caso	$\Theta(n^2)$
Trocas	no máximo $n-1$, ou seja, $O(n)$
Espaço auxiliar	$\Theta(1)$
In-place	Sim
Estável	Não, na versão usual

---

1. Ideia

Divide o vetor em uma parte **ordenada** (à esquerda, inicialmente vazia) e uma **não ordenada**. Em cada passagem, encontra o menor elemento da parte não ordenada e o troca com o primeiro elemento dessa parte.

Diferente do Bubble Sort, **não troca a cada comparação**: só faz no máximo **uma troca por passagem**.

Passo a passo em `[5, 2, 4, 1]`

Passagem	Parte não ordenada	Menor	Ação	Vetor
1	`5 2 4 1`	1 (pos 3)	troca com pos 0	`[1, 2, 4, 5]`
2	`2 4 5`	2 (pos 1)	já está no lugar	`[1, 2, 4, 5]`
3	`4 5`	4 (pos 2)	já está no lugar	`[1, 2, 4, 5]`

Mesmo já ordenado após a 1ª passagem, o algoritmo continua procurando o menor em cada parte restante.

---

2. Implementação

```
def selection_sort(v):
    n = len(v)
    for i in range(n - 1):
        menor = i                          # 1. assume que a posição atual tem o menor
        for j in range(i + 1, n):          # 2. percorre a parte não ordenada
            if v[j] < v[menor]:
                menor = j                  # 3. atualiza o índice do menor
        if menor != i:                     # 4. no máximo uma troca por passagem
            v[i], v[menor] = v[menor], v[i]
    return v
```

---

3. Análise de complexidade passo a passo

3.1 Comparações

Na passagem i ($i = 0, \ldots, n-2$), o laço interno vai de $i+1$ a $n-1$: são $n-1-i$ comparações.

$$C(n) = \sum_{i=0}^{n-2} (n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

Esse valor **não depende da entrada**: para achar o menor, é preciso olhar **todos** os elementos restantes, mesmo que o vetor já esteja ordenado.

> **Pergunta da aula:** se o vetor já estiver ordenado, o Selection Sort consegue evitar as comparações? **Não.** Ele ainda precisa procurar o menor em cada parte restante.

3.2 Trocas

No máximo 1 troca por passagem → no máximo $n-1$ trocas: $O(n)$.

- Vetor ordenado: 0 trocas efetivas (com o teste `if menor != i`).
- Pior caso: $n-1$ trocas.

Vantagem: quando **movimentar** elementos é caro (registros grandes), fazer só $O(n)$ trocas pode ser relevante — o Bubble Sort pode fazer até $n(n-1)/2$.

3.3 Tempo total

$$T(n) = c_1 \frac{n(n-1)}{2} + c_2 \cdot (\text{trocas}), \qquad 0 \le \text{trocas} \le n-1$$

O termo quadrático está sempre presente: **melhor = médio = pior =** $\Theta(n^2)$.

3.4 Espaço

Só `i`, `j`, `menor` e o temporário da troca: $\Theta(1)$ → **in-place**.

---

4. Por que não é estável?

A troca de longa distância pode "pular" um elemento por cima de outro igual. Exemplo da aula, `3 1 3 5 2 3`, rotulando os 3 como 3_a, 3_b, 3_c:

Passagem	Menor da parte restante	Troca	Vetor
início	—	—	$3_a\ 1\ 3_b\ 5\ 2\ 3_c$
1	1 (pos 1)	pos 0 ↔ pos 1	$1\ 3_a\ 3_b\ 5\ 2\ 3_c$
2	2 (pos 4)	pos 1 ↔ pos 4	$1\ 2\ 3_b\ 5\ 3_a\ 3_c$ ← 3_a saltou por cima de 3_b
3	3_b (pos 2)	nenhuma	$1\ 2\ 3_b\ 5\ 3_a\ 3_c$
4	3_a (pos 4)	pos 3 ↔ pos 4	$1\ 2\ 3_b\ 3_a\ 5\ 3_c$
5	3_c (pos 5)	pos 4 ↔ pos 5	$1\ 2\ 3_b\ 3_a\ 3_c\ 5$

Resultado: 3_b ficou **antes** de 3_a — a ordem relativa original $(3_a, 3_b, 3_c)$ foi quebrada. **Instável.**

> Uma versão estável é possível: em vez de trocar, **deslocar** os elementos uma posição para a direita e inserir o menor na posição i (mais movimentações).

---

5. Exercícios resolvidos

Exercício 6 — `v = [29, 13, 37, 5, 41, 19, 2, 34, 11, 23]` ($n = 10$)

a/b) Vetor após cada iteração do laço externo, com o menor encontrado e sua posição antes da troca:

i	Comparações	Menor (posição)	Troca efetiva?	Vetor após a iteração
0	9	2 (pos 6)	sim: pos 0 ↔ 6	`[2, 13, 37, 5, 41, 19, 29, 34, 11, 23]`
1	8	5 (pos 3)	sim: pos 1 ↔ 3	`[2, 5, 37, 13, 41, 19, 29, 34, 11, 23]`
2	7	11 (pos 8)	sim: pos 2 ↔ 8	`[2, 5, 11, 13, 41, 19, 29, 34, 37, 23]`
3	6	13 (pos 3)	não (já no lugar)	`[2, 5, 11, 13, 41, 19, 29, 34, 37, 23]`
4	5	19 (pos 5)	sim: pos 4 ↔ 5	`[2, 5, 11, 13, 19, 41, 29, 34, 37, 23]`
5	4	23 (pos 9)	sim: pos 5 ↔ 9	`[2, 5, 11, 13, 19, 23, 29, 34, 37, 41]`
6	3	29 (pos 6)	não	`[2, 5, 11, 13, 19, 23, 29, 34, 37, 41]`
7	2	34 (pos 7)	não	`[2, 5, 11, 13, 19, 23, 29, 34, 37, 41]`
8	1	37 (pos 8)	não	`[2, 5, 11, 13, 19, 23, 29, 34, 37, 41]`

c) Comparações: $9 + 8 + \cdots + 1 = \dfrac{10 \cdot 9}{2} = \mathbf{45}$.

d) Trocas efetivas: 5 (iterações 0, 1, 2, 4 e 5).

e) Melhor, médio e pior caso: $\Theta(n^2)$ (as 45 comparações acontecem sempre).

f) Comparação com o Bubble Sort: o Bubble faria uma troca para cada inversão do vetor — **25 trocas** neste vetor (e até $n(n-1)/2 = 45$ no pior caso). O Selection fez só **5** (e nunca mais que $n - 1 = 9$). Os dois fazem 45 comparações (Bubble sem parada antecipada).

Exercício 1 (parte Selection) — `[7, 3, 5, 2]`

1ª passagem: menor = 7 → 3 → (5 não) → 2 (pos 3); 3 comparações; troca pos 0 ↔ 3 → `[2, 3, 5, 7]`. É o algoritmo que faz **seleção explícita do menor**. Resolução completa em 02 — Fundamentos.

Exercício 7 (parte Selection)

Vetor	Comparações	Trocas
`v1 = [16, 9, 28, 3, 21, 7, 35, 12, 5, 24]`	45	8
`v2` (já ordenado)	45	0

O Selection **não** se beneficia de entrada ordenada (diferente de Insertion e Bubble otimizado).

Matérias correlatas

> **Soma de Gauss** para as $n(n-1)/2$ comparações: apêndice de 01 — Recursividade.

> **Heap Sort** é um "Selection Sort com estrutura de dados": em vez de procurar o maior em tempo linear, usa um heap para obtê-lo em $O(\log n)$, caindo de $\Theta(n^2)$ para $\Theta(n \log n)$. Veja 08 — Heap Sort.

Estabilidade e comparação com os outros quadráticos: 02 — Fundamentos, 03 — Bubble Sort, 05 — Insertion Sort.